

AIM2 Preparation

Week 1 - Day 7: Object-Oriented Programming + Final Project

5-Minute Review Notes | Classes, Objects, Inheritance & Student Grade Sheet Analyzer

Day 7 Goal

Understand the foundations of object-oriented programming (OOP) and combine the entire Week 1 Python foundation into a final student data-processing project.

1. What Is OOP?

Object-Oriented Programming organizes related data and behavior together inside objects. A class acts as a blueprint; objects are individual instances created from that blueprint.

2. Classes and Objects

```
class Student:
    pass

student1 = Student()
```

Student is the class. **student1** is an object (instance) created from that class.

3. `__init__()` Constructor

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

student1 = Student("Krishna", 28)
```

`__init__()` runs automatically when an object is created. It initializes the object's attributes.

4. Understanding self

```
self.name = name
self.age = age
```

`self` refers to the current object. It allows each object to store and access its own data.

5. Attributes and Methods

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def display_info(self):
        print(self.brand, self.model)
```

Concept	Meaning
Attribute	Data stored inside an object, e.g. <code>self.name</code>
Method	A function defined inside a class
Object	An instance of a class
Class	Blueprint used to create objects

6. StudentScore Class

```
class StudentScore:
    def __init__(self, name, math, python, ai):
        self.name = name
        self.math = math
        self.python = python
        self.ai = ai

    def total(self):
        return self.math + self.python + self.ai

    def average(self):
        return self.total() / 3
```

This class combines student score data with the calculations that operate on those scores.

7. Grade and Result Methods

```
def grade(self):
    avg = self.average()
    if avg >= 90:
        return "A"
    elif avg >= 80:
        return "B"
    elif avg >= 70:
        return "C"
    else:
        return "F"

def result(self):
    return "Pass" if self.average() >= 50 else "Fail"
```

Methods can call other methods through self, which keeps related calculations together.

8. Inheritance

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Student(Person):
    def __init__(self, name, age, student_id):
        super().__init__(name, age)
        self.student_id = student_id
```

Inheritance allows a child class to reuse and extend a parent class. Student inherits the attributes/behavior of Person.

9. super()

```
super().__init__(name, age)
```

super() gives access to the parent class implementation. Here it calls Person's constructor instead of rewriting the same initialization logic.

10. OOP Mental Model

- Class = blueprint.
- Object = actual instance built from the blueprint.
- __init__ = initialization performed when the object is created.
- self = current object.
- Attribute = data belonging to an object.

- Method = behavior/function belonging to a class.
- Inheritance = child class reuses/extends parent class.
- `super()` = access parent-class functionality.

Week 1 Final Project - Student Grade Sheet Analyzer

Integration Project | Python Fundamentals -> OOP + File Processing

Project Goal

Build a student grade-processing application that reads student information from a text file, performs score calculations, classifies students, and generates class-level summaries.

Project Input

The project used **student_list.txt** containing student information such as:
name, math, python, ai, age

Core Project Workflow

Read file -> Parse records -> Convert values -> Calculate scores -> Classify students -> Build summaries -> Display results

Concepts Combined

- File handling to load student records.
- Strings and type conversion to parse file values.
- Lists to store students and result groups.
- Loops to process every student.
- Conditions to determine grades/results.
- Functions/methods to organize calculations.
- Exception-aware thinking for potentially invalid data.
- OOP to represent student information and behavior.
- Aggregation logic for class-level summaries.

Student-Level Calculations

```
total = math + python + ai
average = total / 3
```

Each student record was processed to obtain total score, average score, grade/result, and other classifications.

Class Summary Features

- Highest scorer.
- Lowest scorer.
- Class total.
- Class average.
- Passed students.
- Failed students.
- Bright students with average ≥ 90 .

Example: Tracking Highest and Lowest

```
highest_student = max(students, key=lambda student: student.average())
lowest_student = min(students, key=lambda student: student.average())
```

Conceptually, the program compares student averages to determine top and bottom performance. The exact implementation can also be written using loops.

Example: Passed / Failed Lists

```

if student.average() >= 50:
    passed_students.append(student.name)
else:
    failed_students.append(student.name)

```

Example: Bright Students

```

if student.average() >= 90:
    bright_students.append(student.name)

```

This turns a business/analysis rule into a simple Boolean condition.

Why the Final Project Matters

The final project was not just an OOP exercise. It integrated the full Week 1 foundation into one practical data-processing workflow.

Week 1 Skill	How It Appeared in the Project
Variables / data types	Student fields and calculated values
Lists / dictionaries	Collections and structured data
Control flow	Grades, pass/fail, bright-student rules
Functions	Reusable calculations
Loops	Processing multiple student records
Files	Reading student_list.txt
Exceptions	Safer handling of invalid inputs/data
Modules	Reusable code organization
OOP	Student classes, attributes and methods

Day 7 Quick Syntax Cheat Sheet

```

class Student:
    def __init__(self, name):
        self.name = name

    def display(self):
        print(self.name)

student = Student("Krishna")
student.display()

class Student(Person):
    def __init__(self, name, age):
        super().__init__(name, age)

```

Final Week 1 Takeaways

- OOP groups related data and behavior into classes and objects.
- `__init__` initializes object attributes.
- `self` refers to the current object.
- Methods define what an object can do.
- Inheritance reduces duplication by reusing parent-class behavior.
- `super()` accesses parent-class functionality.
- The final project combined files, loops, conditions, functions, data structures, and OOP.
- Week 1 established the Python foundation needed for NumPy, Pandas, data analysis, and ML.

Week 1 Completion

Week 1 - Python Fundamentals: 7/7 COMPLETE

Final Project: Student Grade Sheet Analyzer

Progression: Fundamentals -> Data Structures -> Control Flow -> Functions -> Loops -> Files/Exceptions/Modules
-> OOP + Integrated Project